Studio Nx

Backend(FastAPI/Python) 42 — POST · DELETE /api/v1/users/me/profile-image

2026-10-07

Backend(FastAPI/Python) 42 — `POST · DELETE /api/v1/users/me/profile-image`

항목	내용
파일	`studionx/backend/app/api/v1/users.py` — `upload_profile_image()` · `delete_profile_image()`
작업	P1-08 회원 정보 — 프로필 사진 올리기 · 지우기 · PRD 01 FR-106
작성	Claude (AI) · 2026-10-07 · 사용자 검토
이전 · 다음	41 `GET · PATCH /users/me` · 43 `GET /profile-images/{name}`

1. 하는 일

요청	응답
`POST` multipart, 칸 `file` = JPG(1.2MB)	200 내 정보 — `"profile_image_url": "/api/v1/profile-images/3f9c….jpg"`
`POST` GIF	415 `UNSUPPORTED_FILE_TYPE`
`POST` 2MB 초과	413 `FILE_TOO_LARGE`
`POST` 칸 이름이 `file` 이 아님 (`photo`)	422 `VALIDATION_ERROR`
`DELETE`	200 내 정보 — `"profile_image_url": null`, 파일 삭제
`DELETE` (사진 없음 · 두 번)	200
토큰 없음	401 — 파일을 저장하지 않음

2. 언제 쓰나

화면 (userInfo.html, P1-10)	부르는 것
Upload photo → 파일 고르기	`FormData` 에 `file` 을 넣어 `api.post('/users/me/profile-image', formData)` → 응답의 `profile_image_url` 로 `<img>` 바꾸기
Remove	`api.delete('/users/me/profile-image')` → 첫 글자 동그라미로

함수들이 이어지는 모습 (`POST`)

순서	하는 곳	일
①	06 `CurrentUser`	토큰 → 활동 회원 (아니면 401, 파일은 읽지도 않음)
②	FastAPI `UploadFile`	multipart 의 `file` 칸 (없으면 422)
③	38 `save_profile_image`	종류 · 크기 · 내용 검사 → 저장 → 예전 파일 삭제
④	36 `to_me_response`	새 사진 주소(35)가 담긴 응답

3. 코드

import (파일 맨 위 — 41 과 같음, 추가 없음)

함수 (경로 · 응답 설정은 템플릿 그대로)

```
@router.post(
    "/me/profile-image",
    response_model=MeResponse,
    responses={413: {"model": ErrorResponse}, 415: {"model": ErrorResponse}},
)
async def upload_profile_image(file: UploadFile, user: CurrentUser, db: Db) -> MeResponse:
    user = await user_service.save_profile_image(db, user, file)
    return user_service.to_me_response(user)


@router.delete("/me/profile-image", response_model=MeResponse)
def delete_profile_image(user: CurrentUser, db: Db) -> MeResponse:
    return user_service.to_me_response(user_service.delete_profile_image(db, user))
```

문법 풀이

식	뜻
`file: UploadFile`	multipart 본문의 `file` 칸을 파일로 받음. 큰 파일은 디스크 임시 파일에 두어 메모리를 아낌. `python-multipart` 패키지가 필요 (requirements 에 있음)
`async def` · `await`	38 이 `await file.read(…)` 를 쓰므로 라우터도 비동기
`user = await …`	매개변수 `user` 에 저장 결과(같은 회원, 새 경로)를 다시 넣음

4. 흐름

POST: 토큰 확인 → `file` 칸 받기 → 38 로 검사 · 저장 → 36 응답. **DELETE:** 토큰 확인 → 39 로 지우기 → 36 응답.

5. 왜 이렇게

선택	이유
라우터는 연결만	검사 · 저장 순서 · 실패 처리는 38 · 39 에서 시험을 마쳤다
응답 = 내 정보 전체	화면이 새 사진 주소를 바로 받아 `<img src>` 를 바꾼다 (다시 `GET` 불필요)
사진이 없어도 `DELETE` 200	39 와 같음 — 두 번 눌러도 사용자에게 오류를 보이지 않는다
토큰 검사가 파일보다 먼저	로그인하지 않은 사람이 파일을 올려도 저장되지 않는다 (시험으로 확인: `profiles` 폴더가 생기지도 않음)

알아 둘 점 — 비동기 안의 DB: `upload_profile_image` 는 `async` 인데, 38 안의 DB 저장(`commit`)은 동기 방식이라 그동안 서버의 다른 요청이 아주 잠깐 기다린다. 사진 저장은 한 번에 수 밀리초라 Phase 1 에서는 문제없다. 사용자가 많아지면 파일만 먼저 읽고 DB 일은 동기 함수로 나누면 된다.

6. 시험 (`tests/test_user_routes.py` 에 7개 추가 · 모두 18개, 실제 앱 + MySQL + 임시 폴더)

시험	확인한 것
JPG 올리기	200, 주소 모양 `/api/v1/profile-images/{32자}.jpg`, 그 파일 내용이 올린 것과 같음
GIF · 2MB 초과	415 `UNSUPPORTED_FILE_TYPE` · 413 `FILE_TOO_LARGE`
칸 이름 `photo`	422 `VALIDATION_ERROR`
토큰 없이 올리기	401, 저장 폴더가 생기지도 않음
지우기	200, 주소 `null`, 파일 삭제 → 한 번 더 지우기도 200
토큰 없이 지우기	401

전체 시험 306개 통과, `ruff` 검사 통과. 진짜 `storage/` 폴더는 비어 있음(`.gitkeep` 만).